Instituto Politécnico Nacional
Centro de Investigación en Computación

Teoría de la Computación

Nombre Completo: Mondragón Vega Fernando Iván
Fecha de entrega: 4 de Noviembre de 2026

TAREA No. 4

1. You are consulting for a trucking company that does a large amount of business shipping packages between New York and Boston. The volume is high enough that they have to send a number of trucks each day between the two locations. Trucks have a fixed limit W on the maximum amount of weight they are allowed to carry. Boxes arrive at the New York station one by one, and each package i has a weight w_i. The trucking station is quite small, so at most one truck can be at the station at any time. Company policy requires that boxes are shipped in the order they arrive; otherwise, a customer might get upset upon seeing a box that arrived after his make it to Boston faster. At the moment, the company is using a simple greedy algorithm for packing: they pack boxes in the order they arrive, and whenever the next box does not fit, they send the truck on its way.
   But they wonder if they might be using too many trucks, and they want your opinion on whether the situation can be improved. Here is how they are thinking. Maybe one could decrease the number of trucks needed by sometimes sending off a truck that was less full, and in this way allow the next few trucks to be better packed.
   Prove that, for a given set of boxes with specified weights, the greedy algorithm currently in use actually minimizes the number of trucks that are needed. Your proof should follow the type of analysis we used for the Interval Scheduling Problem: it should establish the optimality of this greedy packing algorithm by identifying a measure under which it "stays ahead" of all other solutions.
   Solution
   Let's assume we have n boxes with weights $w_1, w_2, ..., w_n$ that are packed in trucks with weight limit W. The boxes have to be packed in order of arrival. Let G_T be the weight carried by truck T using the greedy algorithm and H_T the weight carried by T using another solution.

   Claim. After packing every truck T, the greedy algorithm has packed the same amount or more boxes than any other solution.

   Proof. (By induction on T)
   Base case. ($T = 1$) The greedy algorithm packs as many boxes as possible, so $G_1 = w_1 + ... + w_j$. Whereas other solutions can pack up to the same amount, otherwise the truck will be overloaded, that is $H_1 = w_1 + ... + w_i$ where $i \leq j$. And $H_1 \leq G_1 \leq W$.
   Inductive step. Let's assume this property holds up to truck $T = k - 1$. In the other

solution, there have been fitted i' boxes, while the greedy algorithm has fitted j' boxes. With the other soultion, truck k must fit packages with total weight $H_k = w_{i'+1}, ..., w_i$. Since $i' \leq j'$, using the greedy algorithm, we can fit at least packages with weights $G_k = w_{j'+1}, ..., w_i$. Here, we note that $G_k \leq H_k$, so the truck G_k can possibly fit more boxes on the condition that the next package has weight $w_{i+1} \leq H_k - G_k = w_{i'+1} + ... + w_{j'}$. This means, with the Greedy algorithm we can fit at least the same amount of boxes or even more than any other algorithm.

2. Suppose you are given a connected graph G, with edge costs that are all distinct. Prove that G has a unique minimum spanning tree.
Solution (By contradiction)
Let T, T' be two different minimum spanning trees in G. By definition of MST, both must include every node in G, so they must have at least one edge that is different. Let f be such an edge, *i.e.* $f \in T', f \notin T$.
Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then the MST contains e.
Let S' be a cutset containing f. We have two cases:

   - If f is the min cost edge, since T is a MST, then f must be in T. But that is a contradiction.
   - If f is not the min cost edge in S', then T' is not a MST.

   Since both options lead to a contradiction, then there's no such an edge f and $T = T'$.

3. One of the basic motivations behind the Minimum Spanning Tree Problem is the goal of designing a spanning network for a set of nodes with minimum *total* cost. Here we explore another type of objective: designing a spanning network for which the *most expensive* edge is as cheap as possible.
Specifically, let $G = (V, E)$ be a connected graph with n vertices, m edges, and positive edge costs that you may assume are all distinct. Let $T = (V, E')$ be a spanning tree of G; we define the *bottleneck edge* of T to be the edge of T with the greatest cost.
A spannning tree T of G is a *minimum-bottleneck spanning tree* if there is no spanning tree T' of G with a cheaper bottleneck edge.

   a) Is every minimum-bottleneck tree of G a minimum spanning tree of G? Prove of give a counterexample.

   b) Is every minimum spanning tree of G a minimum-bottleneck tree of G? Prove or give a counterexample.

   Solution

   a) By counterexample, consider the connected graph G with vertices $V = \{a, b, c, d\}$ and edges $E = \{(a, b) = 2, (a, c) = 4, (a, d) = 1, (b, d) = 3\}$. It's MST $S = (V, E')$ where $E' = \{(a, b), (a, c), (a, d)\}$ and has total weight $W = 7$. On the other hand,

consider $B = (V, E'')$ where $E'' = \{(a, c), (a, b), (b, d)\}$ (the path c, a, b, d). By definition, B is a minimum bottleneck tree, since there is no spanning tree that does not contain the edge $(a, c) = 4$ (a cheaper bottleneck edge). However, $S \neq B$, and thus the claim is disproven.

b) Proof. (By contradiction)
Let S be a minimum spanning tree of G that is not a minimum-bottleneck tree of G. Let e be the bottleneck edge of S. If S is not a MBT, then by definition there is another spanning tree S' with a cheaper bottleneck edge than e. However, by the cut property if $e \in S$ that's because there is no other edge with less cost than e in any cutset that includes e. A contradiction.

4. A group of network designers at the communications company CluNet find themselves facing the following problem. They have a connected graph $G = (V, E)$, in which the nodes represent sites that want to communicate. Each edge e is a communication link, with a given available bandwidth b_e.
For each pair of nodes $u, v \in V$, they want to select a single $u - v$ path P on which this pair will communicate. The *bottleneck rate* $b(P)$ for this path P is the minimum bandwidth of any edge it contains; that is, $b(P) = \min_{e \in P} b_e$. The *best achievable bottleneck rate* for the pair u, v in G is simply the maximum, over all $u - v$ paths P in G, of the value $b(P)$.
It's getting to be very complicated to keep track of a path for each pair of nodes, and so one of the network designers makes a bold suggestion: Maybe one can find a spanning tree T of G so that for *every* pair of nodes u, v, the unique $u - v$ path in the tree actually attains the best achievable bottleneck rate for u, v in G. (In other words, even if you could choose any $u - v$ path in the whole graph, you couldn't do better than the $u - v$ path in T.)
This idea is roundly heckled in the offices of CluNet for a few days, and there's a natural reason for the skepticism: each pair of nodes might want a very different-looking path to maximize its bottleneck rate; why should there be a single tree that simultaneously makes everybody happy? But after some failed attempts to rule out the idea, people begin to suspect it sould be possible.
Show that such a tree exists, and give an efficient algorithm to find one. That is, give an algorithm constructing a spanning tree T in which, for each $u, v \in V$, the bottleneck rate of the $u - v$ path in T is equal to the best achievable bottleneck rate for the pair u, v in G.
Solution
Let $b_M = max_{e \in E} b_e$ be the maximum bandwidth that any edge in E has. Start by assigning each edge in G the weight $w_e = b_M - b_e$. Finally, apply any algorithm to find a minimum spanning tree T of this graph.
Claim. For each $u, v \in V$, the bottleneck rate of the $u - v$ path in T is equal to the best achievable bottleneck rate for the pair u, v in G.
Proof (By contradiction). Let $u', v' \in V$ have a path P from $u' - v'$ in T so that it doesn't attain the best achievable bottleneck rate for that pair in G. Then, there exists another path P' in G with a better bottleneck rate. This means that there's at least an edge $f' \notin T$ that improves the bottleneck rate of P' in comparison to P. Note that f'

has the property $w_{f'} = max_{e \in P'} w_e$. Let $f \in P$ be the edge that limits the bottleneck rate in P, that is $w_f = max_{e \in P} w_e$. When we compare those edges, it should be the case that $w_f > w_{f'}$. Now, if we add f' to T, it generates a cycle. Let S be a cutset that includes both f' and f. This is achievable since we can group the nodes in two sets so that u' and v' are not in the same set. By the cut property, we must include f' in T since it is the min cost edge in S. But $f' \notin T$, a contradiction. □